

Gradescope questions

After correctly implementing `pwords` (i.e. passing autograder tests), compare `lwords` and `pwords` by answering the following questions.

- 1 Briefly compare the performance of `lwords` and `pwords` when run on the Gutenberg dataset. How might you explain their relative performance? Simply note and explain what you observe, whether or not the result is what you expected! *Hint: Look into `man time`.*
- 2 Under what circumstances would `pwords` perform better than `lwords`? Under what circumstances would `lwords` perform better than `pwords`? Is it possible to use multithreading in a way that always performs better than `lwords`?

Question 1: Performance Comparison & Explanation

Observation:

When run on the Gutenberg dataset, `pwords` often performs worse than or exhibits no significant speedup over `lwords` in wall-clock time (real time in time). Furthermore, `pwords` consumes significantly higher total CPU time (user + sys time).

Explanation:

1. **Coarse-Grained Lock Contention:** `pwords` protects the entire shared list with a single mutex (`wclist->lock`). Every time a thread needs to look up or add a word, it must hold this lock. Because linked list lookup is an $O(N)$ operation, threads hold the lock for a relatively long time during linear scans. This forces all worker threads to serialize, spending most of their execution time blocked and waiting for the lock.
2. **Thread Management Overhead:** Creating (`pthread_create`), scheduling, context-switching, and joining (`pthread_join`) multiple threads adds non-trivial system overhead (sys time). When threads spend most of their time waiting on lock contention, this overhead hurts total performance rather than helping it.

Question 2: Circumstances & General Multithreading Limits

When `pwords` performs better than `lwords`:

- **I/O-Bound Workloads:** When files are extremely large or stored on slow disk/network storage. Opening, reading, and parsing file streams into words happens concurrently outside the critical section while other threads hold the lock.
- **Many Independent Files on Multi-Core Hardware:** When processing many large files on multi-core CPUs, provided the non-critical section work (file reading/tokenizing) far outweighs the time spent inside the mutex.

When `lwords` performs better than `pwords`:

- **Small Input Files:** For small datasets or single files, the fixed time cost of creating and managing threads exceeds the time required to count the words sequentially.
- **High Lock Contention / High Word Repetition:** When multiple threads continuously attempt to access the exact same shared data structure simultaneously, context-switching overhead slows execution down below single-threaded speed.
- **Single-Core Systems:** On a single CPU core, multi-threading adds synchronization and scheduling overhead without any true physical parallelism.

Is it possible for multithreading to ALWAYS perform better than `lwords`?

No. According to **Amdahl's Law**, every program has sequential overhead (e.g., thread creation, memory allocation, final output sorting). For tiny inputs or single short files, the fixed overhead of thread initialization (`pthread_create` / kernel calls) will strictly exceed the tiny cost of single-threaded execution.

